

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

SHASHANK SHANTHARAM NAYAK (1BM23CS313)

in partial fulfillment for the award of the degree of

**BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Aug-2025 to Jan-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “ Bio Inspired Systems (23CS5BSBIS)” carried out by **SHASHANK SHANTHARAM NAYAK (1BM23CS313)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

Rohith Vaidya K Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	29/8/25	Gene Algorithm	4
2	12/9/25	Particle Swarm Optimization	12
3	10/10/25	Ant Colony Optimization	17
4	17/10/25	Cuckoo Search	22
5	17/10/25	Grey Wolf Optimizer	26
6	07/11/25	Parallel Cell Automaton	31
7	29/8/25	Gene Expression Algorithm	35

Github Link:

<https://github.com/DevelSSN/1BM23CS313-SHASHANK-SHANTHARAM-NAYAK/tree/BIS>

Program 1

Gene Algorithm

Design and implement a simple genetic algorithm (GA) to iteratively evolve a population of candidate solutions (chromosomes) toward minimizing a given mathematical fitness function $f(x)=x^2$.

Algorithm:

29/8/25

Genetic Algorithms (GA)

Phases

- Initialization
- Fitness assignment
- Selection
- Crossover
- Termination

Initialization:

Initialize chromosomes to binary strings

Fitness assignment:

Take a function that measures the best chromosomes

$$f(x) = x^2$$

String No	Initial Population	Value (x)	f(x)	Prob $\frac{f(x)}{\sum f(x)}$	%Prob	Expected Count	$E(P(x))$
1	01100	12	144	0.1267	12.67	0.5068	1 ✓
2	11001	25	625	0.5411	54.11	2.1644	2 ✓
3	00101	5	25	0.0216	2.16	0.0864	0 ✗
4	10011	19	361	0.3125	31.25	1.25	1 ✓

$$\sum P(x) = 1155$$

$$n = 4$$

$$\text{Avg}(f(x)) = \frac{1155}{4} = 288.75 \quad \text{Max} = 625$$

$$\text{Expected Count} = \frac{f(x)}{\text{Avg}(f(x))} = P(x) \cdot n$$

~~Count~~ Population in mating pool is 1 → 1, 2 → 2, 3 → 0, 4 → 1

Mating Pool

String No	Mating Pool	Crossover Point	Offspring crossover	Value x	Fitness $f(x) = x^2$
1	01100	4	01101	13	169
2	11001		11000	24	576
3	11001	2	11011	27	729
4	10011		10001	17	289

Max = 729

Crossover:

Single Point H100 10101
 2nd point 11 100 10 101 $\rightarrow C_1, C_2$
 swap

Two point : 0 | 101 | 0 C_1, C_2
 1 | 100 | 1 swap

Uniform: Each bit has 50% chance to get from each C_1, C_2

Mutation $\downarrow$ Random

Str	Offspring crossover	Mutation strings	Offspring Mutation	Value x	Fitness
1	01101	10000	11101	29	841
2	11000	00000	11000	24	576
3	11011	00000	11011	27	729
4	10001	00101	10100	20	400

$$x \wedge y = \text{Ans}$$

Max

Round 1 (Max values)

625 $\rightarrow$ 729 $\rightarrow$ 841

Pseudocode

def main:

Input chromosomes

Input mutation chromosomes

while var != 0:

Find max

Crossover

Mutate

Find mutation_max & compute var

Enter chromosomes: 3 5 6 7

Enter mutation - values: 8 3 1 4

Generation 1:

Chromosomes: [3 5 6 7] Expectation: [0 1 1 2]

Selected Parent: [6 7 5 7]

Children after crossover: [7 6 7 5]

Mutants after Mutation: [5 5 6 1]

Generation 2:

Chromosomes: [5 5 6 1], Expectation: [1 1 & 0]

Selected Parent: [5 6 6 5]

Children after crossover: [6 5 5 6]

Mutants after Mutation: [4 6 4 2]

Convergence reached. Exiting

Code:

```
import random
```

```
# Initialize chromosomes and mutation parameters
```

```
def init():
```

```
    global chromosomes
```

```
    global n
```

```
    global mutation
```

```
    ch = input("Enter chromosomes (space separated): ")
```

```
    chromosomes = [int(x) for x in ch.split()]
```

```
    n = len(chromosomes)
```

```
    ch = input("Enter mutation values (space separated): ")
```

```
    mutation = [int(x) for x in ch.split()]
```

```
# Ensure mutation length matches the number of chromosomes
```

```
if len(mutation) < n:
```

```
    # Extend mutation list if too short
```

```
    mutation.extend([mutation[-1]] * (n - len(mutation)))
```

```
elif len(mutation) > n:
```

```
    mutation = mutation[:n] # Truncate mutation list if too long
```

```
# Fitness function:  $f(x) = x^2$  (minimize this)
```

```
def fitness(x):
```

```
    return x**2
```

```
# Compute fitness scores and selection probabilities
```

```
def compute():
```

```
    global probs
```

```
    global expectation
```

```
    fit_scores = [fitness(chromosome) for chromosome in chromosomes]
```

```
    max_val = max(fit_scores)
```

```
    total = sum(fit_scores)
```

```
    probs = [score / total for score in fit_scores]
```

```
# Expectation based on fitness probabilities
```

```
    expectation = [int(round(prob * n)) for prob in probs]
```

```
    print("Chromosomes:", chromosomes, end=", ")
```

```
    print("Expectation:", expectation)
```

```
    return max_val
```

```
# Crossover operation using bitwise masks
```

```
def mix(x, y):
```

```
    mask_high = 24 # 0b11000
```

```
    mask_low = 7 # 0b00111
```

```
    c1 = (x & mask_high) | (y & mask_low)
```

```
c2 = (y & mask_high) | (x & mask_low)
return c1, c2
```

```
# Perform crossover based on selection probabilities
```

```
def cross():
```

```
    global li
```

```
    li = []
```

```
    # Selection based on expectation: replicate chromosomes based on their expectation value
    for i in range(len(chromosomes)):
        li.extend([chromosomes[i]] * expectation[i])
```

```
    random.shuffle(li)
```

```
    print("Selected Parents:", li)
```

```
    temp_li = []
```

```
    # Crossover: create offspring from selected chromosomes
```

```
    for i in range(0, len(li) - 1, 2):
```

```
        c1, c2 = mix(li[i], li[i + 1])
```

```
        temp_li.extend([c1, c2])
```

```
    # If odd number of chromosomes, add last chromosome without crossover
```

```
    if len(li) % 2 != 0:
```

```
        temp_li.append(li[-1])
```

```
    li = temp_li
```

```
    print("Children after Crossover:", li)
```

```
# Apply mutation: XOR mutation based on the provided mutation array
```

```
def mutate():
```

```
    global li
```

```
    global mutation
```

```
    global chromosomes
```

```
    for i in range(len(li)):
```

```
        # Safeguard for mismatch in length
```

```
        mutation_value = mutation[i % len(mutation)]
```

```
        li[i] = li[i] ^ mutation_value # XOR operation for mutation
```

```
    print("Mutants after Mutation:", li)
```

```
    chromosomes[:] = li # Update chromosomes
```

```
# Main algorithm loop
```

```
def main():
```

```
    init() # Initialize chromosomes and mutation values
```

```
    prev_chromosomes = chromosomes[:] # Store initial chromosomes
```

```
    generations = 1 # Track number of generations
```

```

max_generations = 10 # Set maximum generations to avoid infinite loop
fitness_threshold = 0.01 # Threshold for minimal change in fitness between generations

while generations <= max_generations:
    print(f'Generation {generations}:')
    compute() # Compute fitness of the current population
    cross() # Perform crossover operation
    mutate() # Apply mutation operation

    # Calculate variance as the difference in the best fitness between generations
    # Best chromosome fitness in the current population
    current_max_fitness = fitness(max(chromosomes))
    # Best chromosome fitness in the previous population
    prev_max_fitness = fitness(max(prev_chromosomes))
    variance = abs(current_max_fitness - prev_max_fitness)

    # If the fitness change is small, we assume convergence
    if variance < fitness_threshold:
        print("Convergence reached. Exiting.")
        break

    # Update previous chromosomes and continue
    prev_chromosomes = chromosomes[:]
    generations += 1

    # Check for the maximum generation limit
    if generations > max_generations:
        print("Max generations reached. Exiting.")
        break

if __name__ == '__main__':
    main()

```



```
> py ga.py
Enter chromosomes (space separated): 3 5 6 7
Enter mutation values (space separated): 2 3 1 4
Generation 1:
Chromosomes: [3, 5, 6, 7], Expectation: [0, 1, 1, 2]
Selected Parents: [6, 7, 5, 7]
Children after Crossover: [7, 6, 7, 5]
Mutants after Mutation: [5, 5, 6, 1]
Generation 2:
Chromosomes: [5, 5, 6, 1], Expectation: [1, 1, 2, 0]
Selected Parents: [5, 6, 6, 5]
Children after Crossover: [6, 5, 5, 6]
Mutants after Mutation: [4, 6, 4, 2]
Convergence reached. Exiting.
```

Program 2

Particle Swarm Optimization

Develop a Python program to implement the Particle Swarm Optimization (PSO) algorithm to find the maximum value of the given mathematical function $f(x) = -x^2 + 5x + 20$.

Algorithm:

12/11/21

Particle Swarm Optimization

Initialization

$$f(x) = -x^2 + 5x + 20$$

Input points of particles

Algorithm

- Create a population of particles uniformly distributed over X
- Evaluate each position with $f(x) = -x^2 + 5x + 20$
- If position of particle current position is better than previous best position, update
- Determine the best particle ($f(x)$ value)
- Update velocities

$$v_i^{t+1} = v_i^t + c_1 U_1^t (p_{bi}^t - p_i^t) + c_2 U_2^t (g_b^t - p_i^t)$$
- Move to new position

$$p_i^{t+1} = p_i^t + v_i^{t+1}$$
- Do until a criteria until it satisfies every step from 2nd.

S.I.	Position	Velocity	Best Soln	Best Position	$f(x)$
P_1	1 1	0 0	1000	- -	2
P_2	-1 1	0 0	1000	- -	2
P_3	0.5 -0.5	0 0	1000	- -	0.5
P_4	1 -1	0 0	1000	- -	2
P_5	0.25 0.25	0 0	1000	- -	0.125

-	Init	Position	Velocity	Best	Best Position	$f(x)$
	S1	x y	x y	Soln.	x y	
	P ₁	1 1	-0.75 -0.75	2	1 1	2
	P ₂	-1 1	1.25 -0.75	2	-1 1	2
	P ₃	0.5 -0.5	-0.25 0.75	0.5	0.5 -0.5	0.5
	P ₄	1 -1	-0.75 1.25	2	1 -1	2
	P ₅	0.25 0.25	0 0	0.125	0.25 0.25	0.125

-	P1	Position	Velocity	Best	Best Pos	$f(x)$
		x y	x y	Soln	x y	
	P ₁	0.25 0.25	-0.375 -0.375	2	1 1	0.125
	P ₂	0.25 0.25	.625 .375	2	-1 1	0.125
	P ₃	0.25 0.25	-.125 .375	0.5	0.5 -0.5	0.125
	P ₄	0.25 0.25	-.375 .625	2	1 -1	0.125
	P ₅	0.25 0.25	0 0	0.125	0.25 0.25	0.125

Output:

Enter no. of particles: 9

Enter positions of particles: -9.6 -6 -2.6 -1.1 0.6 2.3
2.8 8.3 10

Global best solution: 2.44053 with fitness: 26.24646

Code:

```
import random
from math import sqrt
```

```
c1, c2 = 1, 1
```

```
def fitness(x):
    return -x**2 + 5*x + 20
```

```
def init():
    n = int(input("Enter no. of particles: "))
    v = [0 for i in range(n)]
    x = list(map(float, input("Enter positions of particles:").split()))
    p = x.copy()
    fp = [fitness(xi) for xi in x]
    return n, v, fp, p, x
```

```
def find(n, fp, p):
    max_fitness = float('-inf')
    pos = -1
    for i in range(n):
        if fp[i] > max_fitness:
            max_fitness = fp[i]
            pos = i
    return pos
```

```
def update(n, v, fp, p, x, max_pos):
    r1, r2 = sqrt(random.random()), sqrt(random.random())

    for i in range(n):
        v[i] = v[i] + c1 * r1 * (p[i] - x[i]) + c2 * r2 * (p[max_pos] - x[i])
        x[i] = x[i] + v[i]

    for i in range(n):
        fp[i] = fitness(x[i])
        if fp[i] > fitness(p[i]):
            p[i] = x[i]
```

```
def print_state(v, fp, p, x):
    print(f"""
    {x}
    {p}
    {v}
    {fp}
    """)
```

```
n, v, fp, p, x = init()
print_state(v, fp, p, x)
max_pos = find(n, fp, p)
gbest = p[max_pos]

while True:
    update(n, v, fp, p, x, max_pos)
    max_pos = find(n, fp, p)
    if fitness(gbest) == fitness(p[max_pos]):
        break
    print_state(v, fp, p, x)
    gbest = p[max_pos]

print(f"Global Best Solution: {gbest} with fitness: {fitness(gbest)}")
```

Program 3

Ant Colony Optimization for TSP

Implement the Ant Colony Optimization (ACO) algorithm to find the shortest possible route that visits a set of cities exactly once and returns to the starting city - Traveling Salesman Problem (TSP).

Algorithm:

10/10/25

SURYA Gold

Date _____ Page _____

ACO for TSP

Pseudocode

- Initialize distance matrix, α , β , ρ , no_ants, no_iterations
- Initialize a pheromone matrix to small values
- ~~Init~~ Initialize heuristic matrix visibility = $1/\text{distance}$ for all pairs.

- Init best_tour = None, best_length = ∞
- for no_iterations:

 i. for each ant

 initialize tour = random city

 visited = {start}

 for each city in tour

 select next city by

 (i) Pheromone $^{\alpha}$

 (ii) visibility $^{\beta}$

 add to tour & visited

 ii Calculate tour-length

 iii If tour-length < best length

 best_tour = tour

 best_length = tour-length

MG
10/10/25

 iv. Evaporate pheromones

 pheromone $\times = (1 - \rho)$

 For each ant

 Deposit pheromone $\div = 1/\text{tour-length}$

- Output:

$[(0,0), (1,2), (3,1), (4,3), (2,4), (5,5)]$

Cities: 6

Ants: 10

Iterations: 100

Best tour: [4 1 0 2 3 5]

Tour length: 15.268827

Code:

```
import random
```

```
import numpy as np
```

```
# Helper function to calculate distance between two cities
```

```
def distance(city1, city2, coordinates):
```

```
    # Assuming coordinates are (x, y) pairs
```

```
    x1, y1 = coordinates[city1]
```

```
    x2, y2 = coordinates[city2]
```

```
    return np.sqrt((x2 - x1)**2 + (y2 - y1)**2)
```

```
# Initialize pheromone matrix and visibility matrix
```

```
def initialize_matrices(num_cities, initial_pheromone_value, coordinates):
```

```
    pheromone = np.full((num_cities, num_cities), initial_pheromone_value)
```

```
    visibility = np.zeros((num_cities, num_cities))
```

```
    # Calculate visibility (1 / distance)
```

```
    for i in range(num_cities):
```

```
        for j in range(num_cities):
```

```
            if i != j:
```

```
                visibility[i][j] = 1 / distance(i, j, coordinates)
```

```
    return pheromone, visibility
```

```
# Calculate probability distribution for the next city
```

```
def calculate_probability(current_city, unvisited_cities, pheromone, visibility, alpha, beta):
```

```
    total = 0
```

```
    probabilities = {}
```

```
    for city in unvisited_cities:
```

```
        pheromone_term = pheromone[current_city][city] ** alpha
```

```
        visibility_term = visibility[current_city][city] ** beta
```

```
        probability = pheromone_term * visibility_term
```

```
        total += probability
```

```
        probabilities[city] = probability
```

```
    # Normalize the probabilities
```

```
    for city in probabilities:
```

```
        probabilities[city] /= total
```

```
    return probabilities
```

```
# Ant Colony Optimization (ACO) algorithm
```

```
def ant_colony_optimization(num_cities, coordinates, num_ants, num_iterations, alpha=1, beta=2,  
rho=0.5, initial_pheromone_value=0.1):
```

```
    # Initialize pheromone and visibility matrices
```

```
    pheromone, visibility = initialize_matrices(num_cities, initial_pheromone_value, coordinates)
```

```
    best_tour = None
```

```
    best_length = float('inf')
```



```

# Start iterations
for iteration in range(num_iterations):
    all_tours = []
    all_lengths = []

    # Each ant constructs a tour
    for ant in range(num_ants):
        tour = [random.randint(0, num_cities - 1)] # Start from a random city
        visited_cities = {tour[0]} # Keep track of visited cities

        while len(tour) < num_cities:
            current_city = tour[-1]
            unvisited_cities = set(range(num_cities)) - visited_cities
            probabilities = calculate_probability(current_city, unvisited_cities, pheromone, visibility,
alpha, beta)

            # Decide which city to visit next
            if random.random() < 0.9: # q0 is 0.9 for greedy selection
                next_city = max(probabilities, key=probabilities.get) # Greedy: Choose city with highest
probability
            else:
                next_city = random.choices(list(probabilities.keys()), list(probabilities.values()))[0]

            tour.append(next_city)
            visited_cities.add(next_city)

        # Calculate tour length
        tour_length = 0
        for i in range(num_cities - 1):
            tour_length += distance(tour[i], tour[i + 1], coordinates)
        tour_length += distance(tour[-1], tour[0], coordinates) # Return to the starting city

        all_tours.append(tour)
        all_lengths.append(tour_length)

    # Update best solution
    if tour_length < best_length:
        best_tour = tour
        best_length = tour_length

# Evaporate pheromones
pheromone *= (1 - rho)

# Deposit pheromones from each ant
for ant in range(num_ants):
    tour = all_tours[ant]
    tour_length = all_lengths[ant]
    for i in range(num_cities - 1):
        pheromone[tour[i]][tour[i + 1]] += 1 / tour_length
    pheromone[tour[-1]][tour[0]] += 1 / tour_length # Return to the starting city

```



```

    return best_tour, best_length

# Example usage
if __name__ == "__main__":
    # Example cities coordinates (x, y)
    coordinates = [
        (0, 0), (1, 2), (3, 1), (4, 3), (2, 4), (5, 5)
    ]

    num_cities = len(coordinates)
    num_ants = 10
    num_iterations = 100

    print(coordinates)
    print("Cities:", num_cities)
    print("Ants:", num_ants)
    print("Iterations:", num_iterations)
    alpha, beta, rho = 1, 2, 0.5
    best_tour, best_length = ant_colony_optimization(num_cities, coordinates, num_ants,
num_iterations)
    print(f"Best tour: {best_tour}")
    print(f"Best tour length: {best_length}")

```

```

lab/10-10-25 BIS ? > python aco.py
[(0, 0), (1, 2), (3, 1), (4, 3), (2, 4), (5, 5)]
Cities: 6
Ants: 10
Iterations: 100
Best tour: [4, 1, 0, 2, 3, 5]
Best tour length: 15.26882723033592

```

Program 4

Cuckoo Search

Implement a Levy Flight–based stochastic optimization algorithm to solve a 0/1 Knapsack Problem, maximizing total profit while respecting the knapsack capacity constraint.

Algorithm:

17/10/25

SHRVA Gird

Date

Page

CSA (Cuckoo Search)

Algorithm

```
generate initial population of host nests
evaluate initial population
rank the host nests
while  $g \leq \text{Max Gen}$  do
  for  $i = 1:n$  do
    Carry Levy flights to get new nests
    evaluate new nest
    if new nest  $\gg$  better than old nest then
      replace old nest
    end if
  end for
  a fraction of old population are abandoned and
  new ones are built
  rank the solutions; find current best
  update global best if required
end while
return global best
```

Output

Enter n: 5

Enter values and weights

10 20 30 40 50

5 4 3 2 1

Enter knapsack size: 8

Enter iters: 1000

[0.60617772 0.99281815 0. 1.] [75.90599766] * 5

M9
17/10/25

Code:

```
import numpy as np
from scipy.stats import levy
import random

def init():
    n = int(input("Enter n: "))
    print("Enter values and weights:")
    # Convert input to list of integers
    c = np.array(list(map(int, input().split())))
    a = np.array(list(map(int, input().split())))
    b = int(input("Enter knapsack size: "))
    b = np.array([b for _ in range(n)]) # Ensure b is an array of size n
    maxt = int(input("Enter iters: "))
    return n, c, a, b, random.random(), maxt

def fitness(c, x, a, b, n):
    # Perform element-wise operation
    # Ensure b is a numpy array to allow element-wise subtraction
    temp = np.dot(a, x) - b
    rho = max(c[i] / a[i] for i in range(n))
    return np.dot(c, x) - (rho * temp) ** 2

def Levy(alpha, n):
    step = levy.rvs(size=n)
    return alpha * step

n, c, a, b, p, maxt = init()
x = np.random.randint(2, size=n)
best_x = x.copy()
best_fitness = fitness(c, x, a, b, n)
for t in range(maxt):
    for i in range(n):
        alpha = 0.01
        step = Levy(alpha, n)

        # Added size=n to ensure vector length matches
        new_x = x + np.multiply(step, np.random.choice([1, -1], size=n))
        new_x = np.clip(new_x, 0, 1)
        new_fitness = fitness(c, new_x, a, b, n)
        if sum(new_fitness) > sum(best_fitness):
            best_fitness = new_fitness
            best_x = new_x.copy()

print(best_x, best_fitness)
```

```
lab/10-17-25 BIS ? × python cuckoo.py
Enter n: 5
Enter values and weights:
10 20 30 40 50
5 4 3 2 1
Enter knapsack size: 8
Enter iters: 100
[0.99880404 0.          0.00365066 1.          1.          ] [100.03575311 100.03575311 100.03575311 100.03575311 100.03575311]
```

Program 5

Grey Wolf Optimizer

Implement the Grey Wolf Optimizer (GWO) algorithm to find the global minimum of a complex benchmark function — the Rastrigin function — and analyze its convergence behavior.

Algorithm:

1/10/21

Grey Wolf Optimizer

→ Algorithm

generate initial positions of α, β, δ wolves.

Get Objective Function

Rastrigin

$$20 + x^2 + y^2 - 10(\cos 2\pi x + \cos 2\pi y)$$

Initialize wolves within boundaries (α, β, δ)

α, β, δ are vectors of some dimension, each zero

$\alpha_s, \beta_s, \delta_s$ are scores ∞, ∞, ∞

for every iteration

for every wolf

fitness = obj(wolf)

If fitness < α_s

Shift $\alpha \rightarrow \beta, \beta \rightarrow \delta$

$\alpha_s, \alpha = \text{fitness, wolf}$

else if fitness < β_s

Shift $\beta \rightarrow \delta$

$\beta_s, \beta = \text{fitness, wolf}$

else if fitness < δ_s

$\delta_s, \delta = \text{fitness, wolf}$

$a = 2 - (2E/\text{max-iter})$

for each wolf and dimension

$r_1, r_2 = \text{random, random} [0, 1]$

$A_1 = 2ar_1 - a$

$C_1 = 2r_2$

$D_\alpha = |C_1 - \alpha_j - \text{wolves}[i][j]|$

$x_1 = \alpha_j - A_1 D_\alpha$

} Based on α

Similarly, do for β, δ
wolves $[i][j] = (x_1 + x_2 + x_3) / 3$

Return α, β .

Output:

Best position $[6.62634 \times 10^{-10} \quad 1.791317 \times 10^{-9}]$

Best fitness : 0.0

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Objective function: Rastrigin function
def rastrigin(position):
    x, y = position
    return 20 + x**2 + y**2 - 10 * (np.cos(2 * np.pi * x) + np.cos(2 * np.pi * y))

# Grey Wolf Optimizer
def GWO(obj_func, dim, bounds, num_wolves=10, max_iter=100):
    # Initialize positions of wolves
    wolves = np.random.uniform(bounds[0], bounds[1], (num_wolves, dim))

    # Initialize alpha, beta, and delta wolves
    alpha, beta, delta = np.zeros(dim), np.zeros(dim), np.zeros(dim)
    alpha_score, beta_score, delta_score = np.inf, np.inf, np.inf

    convergence_curve = []

    for t in range(max_iter):
        for i in range(num_wolves):
            fitness = obj_func(wolves[i])

            # Update alpha, beta, delta
            if fitness < alpha_score:
                delta_score, delta = beta_score, beta.copy()
                beta_score, beta = alpha_score, alpha.copy()
                alpha_score, alpha = fitness, wolves[i].copy()
            elif fitness < beta_score:
                delta_score, delta = beta_score, beta.copy()
                beta_score, beta = fitness, wolves[i].copy()
            elif fitness < delta_score:
                delta_score, delta = fitness, wolves[i].copy()

        # Update position of each wolf
        a = 2 - 2 * t / max_iter # a decreases linearly from 2 to 0
        for i in range(num_wolves):
            for j in range(dim):
                r1, r2 = np.random.rand(), np.random.rand()
                A1, C1 = 2 * a * r1 - a, 2 * r2
                D_alpha = abs(C1 * alpha[j] - wolves[i, j])
                X1 = alpha[j] - A1 * D_alpha

                r1, r2 = np.random.rand(), np.random.rand()
                A2, C2 = 2 * a * r1 - a, 2 * r2
                D_beta = abs(C2 * beta[j] - wolves[i, j])
                X2 = beta[j] - A2 * D_beta
```

```

        r1, r2 = np.random.rand(), np.random.rand()
        A3, C3 = 2 * a * r1 - a, 2 * r2
        D_delta = abs(C3 * delta[j] - wolves[i, j])
        X3 = delta[j] - A3 * D_delta

        wolves[i, j] = (X1 + X2 + X3) / 3

    # Keep wolves within bounds
    wolves = np.clip(wolves, bounds[0], bounds[1])

    convergence_curve.append(alpha_score)

return alpha, alpha_score, convergence_curve

# Parameters
dim = 2
bounds = [-5.12, 5.12]
num_wolves = 20
max_iter = 100

# Run GWO
best_position, best_score, convergence = GWO(
    rastrigin, dim, bounds, num_wolves, max_iter)

# Results
print("Best position found:", best_position)
print("Best fitness value:", best_score)

# Plot convergence curve
plt.plot(convergence)
plt.title("Convergence Curve of Grey Wolf Optimizer")
plt.xlabel("Iterations")
plt.ylabel("Best Fitness Value")
plt.grid(True)
plt.show()

```

```

lab/10-17-25 BIS ? > python grey.py
Best position found: [6.26341427e-10 1.79131785e-09]
Best fitness value: 0.0

```

Program 6

Parallel Cell Automaton

Develop and analyze a two-dimensional cellular automaton (CA) based on a Rule 30–inspired update rule, to study emergent complex behavior arising from simple binary local interactions.

Algorithm:

Parallel Cell Automaton

→ Algorithm

Initialize cells to some random/uniform values

Add disturbance somewhere if uniform values were set

For every cell:

Get neighbors of cell

Evaluate neighbors using the objective function

Set the state of the cell to new value.

Each time, keep track of the best value possible

For every iteration:

For every cell:

Get neighbors of cell

Evaluate neighbors with obj. function

Set state of cell to new value

Each iteration, track the best value

Print Best value.

→ Output:

3, 4, 5, 5, 8, 9, 9, 12, ... 99.

Best value: 99 found at step 98, row 1.

MG
7/11/25.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

def objective_function(top, left, center, right, bottom):
    # Rule 30 logic, combining neighbors in the 2D grid
    return top ^ (left or right or bottom or center)

# Initialize the 2D grid (binary states)
size_x = 101 # Number of columns
size_y = 101 # Number of rows
steps = 100 # Number of iterations
grid = np.zeros((steps, size_y, size_x), dtype=int)
# grid = np.random.randint(low=0, high=2, size=(steps, size_y, size_x), dtype=int)
# Set the initial state (a single 1 in the center)
grid[0, size_y // 2, size_x // 2] = 1

# Variable to track the best value and the corresponding step
best_value = 0
best_step = 0
best_row = 0

# Iterate to evolve the 2D grid
for t in range(1, steps):
    for i in range(1, size_x - 1): # Update columns (avoiding boundaries)
        for j in range(1, size_y - 1): # Update rows (avoiding boundaries)
            # Get the neighboring states (top, left, center, right, bottom)
            top = grid[t-1, j-1, i]
            left = grid[t-1, j, i-1]
            center = grid[t-1, j, i]
            right = grid[t-1, j, i+1]
            bottom = grid[t-1, j+1, i]
            grid[t, j, i] = objective_function(
                top, left, center, right, bottom)

# Track the best value: sum of '1's in the grid at this step
row_sums = np.sum(grid[t], axis=1) # Sum of each row
# Best sum of '1's in any row for this step
current_best = np.max(row_sums)
print(f"{current_best}", end=" ")
if current_best > best_value:
    best_value = current_best
    best_step = t
    best_row = np.argmax(row_sums) # Row that produced the best value

# Plot the result
plt.imshow(np.sum(grid, axis=0), cmap='binary', interpolation='nearest')
plt.title(
```

```
f'2D Cellular Automaton (Rule 30-like)\nBest value: {best_value} at step {best_step}, row {best_row}'))\nplt.show()
```

```
# Print the best value information
```

```
print(f"\nBest value: {best_value} found at step {best_step}, row {best_row}")
```

```
bmscecse@bmscecse-HP-Elite-Tower-600-G9-Desktop-PC:~/Downloads/1BN23CS313-SHASHANK-SHANTHARAN-NAYAK/11-07-25$ python3 cell.py
3,4,5,5,8,8,9,9,12,12,12,17,16,21,19,20,23,23,25,24,29,25,29,28,31,28,35,33,36,35,37,37,37,40,41,44,43,47,44,45,45,49,48,53,51,52,51,53,51,56,51,56,51,60,51,55,57,57,53,60,51,59,53,59,51,71,53,63,52,63,54,64,63,63,59,67,63,71,67,69,71,73,75,77,79,81,83,85,87,89,91,93,95,97,99,99,
Best value: 99 found at step 98, row 1
```

```
bmscecse@bmscecse-HP-Elite-Tower-600-G9-Desktop-PC:~/Downloads/1BN23CS313-SHASHANK-SHANTHARAN-NAYAK/11-07-25$ python3 cell.py
3,4,5,5,8,8,9,9,12,12,12,12,17,16,21,19,20,23,23,25,24,29,25,29,28,31,28,35,33,36,35,37,37,37,40,41,44,43,47,44,45,45,49,48,53,51,52,51,53,51,56,51,56,51,60,51,55,57,57,53,60,51,59,53,59,51,71,53,63,52,63,54,64,63,63,59,67,63,71,67,69,71,73,75,77,79,81,83,85,87,89,91,93,95,97,99,99,
Best value: 99 found at step 98, row 1
```

Program 7

Gene Expression Algorithm

Design and implement a Gene Expression Programming (GEP) algorithm to evolve mathematical expressions that approximate or exactly reproduce a given target function. The program should automatically discover the functional relationship between input x and output $f(x)$ through genetic operations such as mutation, crossover, and selection.

Algorithm:

29/8/25

SURYA Chola

Date: _____ Page: _____

Lab - 7

Gene Expression Algorithm

- Initialization
- Fitness assignment
- Selection
- Crossover
- Mutation
- Gene Expression
- Termination

$$f(x) = x^2$$

1) Encoding Technique
0 to 31

Fixed chromosomes in size, terminals and functions (+, -, *, /)

2 Initial population

S	Genotype	Phenotype	Value	Fitness	P
1	* x x	x^2	12	144	0.1243
2	+ x x	$2x$	25	625	0.541
3	x	x	5	25	0.0216
4	- x 2	$x-2$	19	361	0.3125

$\Sigma f = 1155$	$F(x)$	$\text{Round}(P(x))$
Avg = 288.75	0.5	1
	2.1	2
	0.03	0
	1.25	1

3) Making Pool

Sl	Genotype	Point of cross over	Offspring	Phenotype
1	* x x	2	* x +	$x^*(x+)$
2	+ x x	1	+ x x	$2x$
3	+ x x	3	+ x -	$x+(x-)$
4	- x 2	1	+ x 2	$x+2$

4) Crossover x Fitness

13	169
24	576
27	729
17	289

$\text{Max}(f(x)) = 729$ Crossover is at random
gene, not bit

4) Mutation

Sl	Offspring	Mutation	Mutated offspring	Phenotype
1	* x +	+ $\rightarrow$ -	* x -	$x^*(x-)$
2	+ x x	None	+ x x	$2x$
3	+ x -	- $\rightarrow$ *	+ x *	$x+(x^*)$
4	+ x 2	None	+ x 2	$x+2$

x	f(x)
29	841
24	576
27	729
20	400

5) Gene expression

$$\sum f(x) = 2546$$

$$\text{Avg} = 636.5$$

$$\text{Max} = 841$$

6) Iterate to convergence

Pseudocode

Start

- Define $f(x)$
- Define param
- Generate population
- Select Pool
- Mutate
- Express gene and evaluate
- Iterate till convergence
- Output Best value

Output:

50/200 Current population:

$$(x * (x + x)) = 7.871656$$

100/200 Current population:

$$(x * (x + x)) = 7.722239$$

150/200 Current population:

$$(x * (x + x)) = 7.66406197$$

200/200 Current population:

$$((x * x) + ((x * x) + x) + x) = 7.346895$$

Code:

```
import numpy as np
```

```
def fitness(x):
```

```
    # return x ** 2
```

```
    # return (x + x * x) / (x ** 3)
```

```
    return (x * x) + (x * x) - (x + x)
```

```
class GEP(object):
```

```
    def __init__(self, grammar, bounds, len_head, iterations, pop_size, probab_crossover, num_trials):
```

```
        self.grammar = grammar
```

```
        self.bounds = bounds
```

```
        self.len_head = len_head
```

```
        self.len_tail = len_head * (len(grammar['TERM'])) + 1
```

```
        self.iterations = iterations
```

```
        self.pop_size = pop_size
```

```
        self.probab_crossover = probab_crossover
```

```
        self.num_trials = num_trials
```

```
        self.population = [Individual(self.random_genome())
```

```
                               for _ in range(self.pop_size)]
```

```
        self.best = sorted(self.population, key=lambda x: x.fitness)[0]
```

```
    def binary_tournament(self):
```

```
        a, b = np.random.choice(self.population, 2)
```

```
        return a if a.fitness < b.fitness else b
```

```
    def point_mutation(self, genome):
```

```
        rate = 1.0 / len(genome)
```

```
        child = list()
```

```
        for i in range(len(genome)):
```

```
            bit = genome[i]
```

```
            if np.random.random() < rate:
```

```
                if i < self.len_head:
```

```
                    selection = self.grammar['FUNC'] if np.random.random(  
                        ) < 0.5 else self.grammar['TERM']
```

```
                    bit = selection[np.random.randint(0, len(selection))]
```

```
                else:
```

```
                    bit = self.grammar['TERM'][np.random.randint(  
                        0, len(self.grammar['TERM']))]
```

```
            child.append(bit)
```

```
        return ''.join(child)
```

```
    def crossover(self, parent1, parent2):
```

```
        if np.random.random() < self.probab_crossover:
```

```
            return ''.join([a if np.random.random() < 0.5 else b for a, b in zip(parent1, parent2)])
```

```
        else:
```

```
            return parent1
```

```

def reproduce(self, selected):
    children = list()
    for a, b in zip(selected[::2], selected[1::2]):
        children.append(Individual(self.point_mutation(
            self.crossover(a.genome, b.genome))))
    return children

def random_genome(self):
    s = list()
    for _ in range(self.len_head):
        selection = self.grammar['FUNC'] if np.random.random(
        ) < 0.5 else self.grammar['TERM']
        s.append(selection[np.random.randint(0, len(selection))])
    for _ in range(self.len_tail):
        s.append(self.grammar['TERM'][np.random.randint(
            0, len(self.grammar['TERM'])])])
    return ".join(s)

@staticmethod
def cost(program):
    errors = 0.0
    if FROM_TABLE:
        for x, y in TABLE:
            expression = program.replace('x', str(x))
            try:
                score = eval(expression)
            except ZeroDivisionError:
                score = float('inf')
            errors += abs(score - y)
        if np.isnan(errors):
            errors = float('inf')
        return errors / len(TABLE)
    else:
        for _ in range(NUM_TRIALS):
            x = BOUNDS[0] + ((BOUNDS[1] - BOUNDS[0]) * np.random.random())
            expression = program.replace('x', str(x))
            try:
                score = eval(expression)
            except ZeroDivisionError:
                score = float('inf')
            errors += np.abs(score - fitness(x))
        return errors / NUM_TRIALS

@staticmethod
def mapping(genome):
    off = 0
    queue = list()
    root = {'node': genome[off]}
    off += 1

```

```

queue.append(root)
while len(queue) > 0:
    current = queue.pop(0)
    if current['node'] in GRAMMAR['FUNC']:
        current['left'] = {'node': genome[off]}
        off += 1
        queue.append(current['left'])
        current['right'] = {'node': genome[off]}
        off += 1
        queue.append(current['right'])
return root

```

```

@staticmethod
def tree_to_string(exp):
    if 'left' not in exp or 'right' not in exp:
        return exp['node']
    left = GEP.tree_to_string(exp['left'])
    right = GEP.tree_to_string(exp['right'])
    if exp['node'] in ['+', '-', '*', '/']:
        return '({0} {1} {2})'.format(left, exp['node'], right)
    else:
        func = {
            'P': 'np.power',
        }.get(exp['node'], None)
        return '({0}({1}, {2}))'.format(func, left, right)

```

```

def run(self):
    for gen in range(1, self.iterations + 1):
        selected = [self.binary_tournament() for _ in range(self.pop_size)]
        children = self.reproduce(selected)
        children = list(sorted(children, key=lambda x: x.fitness))
        self.best = children[0] if children[0].fitness <= self.best.fitness else self.best
        self.population = sorted(
            (children + self.population), key=lambda x: x.fitness)[:self.pop_size]
        if gen % 50 == 0:
            print('{0}/{1} Current population:'.format(gen, self.iterations))
            print(self.best)
        if self.best.fitness == 0.0:
            print('{0}/{1} Current population:'.format(gen, self.iterations))
            print(self.best)
            break
    return self.best

```

```

class Individual(object):
    def __init__(self, genome):
        self.genome = genome
        self.expression = GEP.mapping(genome)
        self.program = GEP.tree_to_string(self.expression)

```

```

        self.fitness = GEP.cost(self.program)

    def __str__(self):
        return '{0} = {1}'.format(self.program, self.fitness)

GRAMMAR = {'FUNC': ['+', '-', '*', '/'], 'TERM': ['x']}
BOUNDS = [1.0, 10.0]
TABLE = [
    [0, 0],
    [1, 1],
    [2, 4],
    [3, 9],
    [4, 16],
    [5, 25],
    [6, 36],
    [7, 49],
    [8, 64],
    [9, 81],
    [10, 100]
]
NUM_TRIALS = 30
FROM_TABLE = False
LEN_HEAD = 30
ITERATIONS = 200
POP_SIZE = 100
PROB_CROSS = 0.85

def main():
    gep = GEP(GRAMMAR, BOUNDS, LEN_HEAD, ITERATIONS,
              POP_SIZE, PROB_CROSS, NUM_TRIALS)
    gep.run()

if __name__ == '__main__':
    main()

```

```
> py gea.py
```

```
50/200 Current population:
```

```
(x * (x + x)) = 7.871656931630147
```

```
100/200 Current population:
```

```
(x * (x + x)) = 7.722239965697489
```

```
150/200 Current population:
```

```
(x * (x + x)) = 7.664061972927878
```

```
200/200 Current population:
```

```
((x * x) + ((x + x) + x) + x)) = 7.346895675415251
```